

Refactor Summary From This Chat

Goal

This refactor sequence was done to remove structural smells and enforce clearer MVC boundaries:

- Model should not render or depend on view/controller internals.
- View should be passive and not drive game flow directly.
- Controller should mediate state changes, input, audio, and persistence.
- Remove unnecessary pattern overhead where it did not add value.

Step 1: Input and Controller Boundary Cleanup

The first pass removed direct reach-through from input handlers into `game.mainMenu`, `game.levelSelect`, and `game.leaderboard`.

Changes:

- `src/main/java/main/controller/facades/IGameActions.java`
- `src/main/java/main/controller/facades/IGameRead.java`
- `src/main/java/main/controller/inputs/KeyboardInputs.java`
- `src/main/java/main/controller/inputs/MouseInputs.java`
- `src/main/java/main/view/GamePanel.java`
- `src/main/java/main/view/GameWindow.java`

Result:

- Keyboard/mouse now call controller ports.
- `GamePanel` became passive and only exposes listener attachment.
- Window focus handling no longer reaches through panel to grab `Game` internals.

Step 2: View Became Callback-Driven

Menu/selection/leaderboard views were changed to call action interfaces instead of mutating game flow directly.

Changes:

- `src/main/java/main/view/states/MainMenu.java`
- `src/main/java/main/view/states/LevelSelect.java`
- `src/main/java/main/view/states/Leaderboard.java`
- `src/main/java/main/view/states/Actions/MainMenuActions.java`
- `src/main/java/main/view/states/Actions/LevelSelectActions.java`
- `src/main/java/main/view/states/Actions/LeaderboardActions.java`
- `src/main/java/main/controller/Game.java` (implements view action interfaces)

Result:

- View no longer calls `setGameState` directly.

- View no longer directly handles leaderboard file loading.
- View emits intent, controller performs transitions.

Step 3: Leaderboard Data Moved Out of View

Leaderboard file parsing/sorting moved out of view code into model-side service objects.

Added:

- `src/main/java/main/model/leaderboard/ScoreEntry.java`
- `src/main/java/main/model/leaderboard/LeaderboardService.java`

Result:

- `Leaderboard` view only renders provided entries.
- File parsing/sorting logic no longer lives in view state.

Step 4: Remove Model Callback to Controller (`LevelManagerHost`)

`LevelManager` stopped calling back into `Game` for transitions/reload.

Changes:

- Deleted `src/main/java/main/model/levels/LevelManagerHost.java`
- Updated `src/main/java/main/model/levels/LevelManager.java`
- Updated `src/main/java/main/model/GameModel.java`
- Updated `src/main/java/main/controller/Game.java`

Result:

- `LevelManager.loadNextLevel()` returns outcome instead of calling controller methods.
- Controller/model flow became explicit and easier to follow.

Step 5: Shared Config Extraction

Constants previously tied to `main.controller.Game` were centralized.

Added:

- `src/main/java/utilities/GameConfig.java`

Updated usages in:

- `src/main/java/main/model/**`
- `src/main/java/utilities/HelpMethods.java`
- `src/main/java/utilities/LoadSave.java`

Result:

- Model/utility layers no longer depend on controller constants.

Step 6: Finish Step 1 Fully (No Rendering in Model)

Remaining rendering/image concerns were moved out of model entities and levels.

Key model changes:

- `src/main/java/main/model/entities/entity/Player.java`
- `src/main/java/main/model/entities/entity/Spike.java`
- `src/main/java/main/model/entities/entity/TriggerPlatform.java`
- `src/main/java/main/model/entities/entity/TriggerSpike.java`
- `src/main/java/main/model/entities/entity/SpawnPlatform.java`
- `src/main/java/main/model/entities/entity/MovingPlatform.java`
- `src/main/java/main/model/entities/entity/DeathSprite.java`
- `src/main/java/main/model/levels/Level.java`
- `src/main/java/main/model/levels/LevelConfigLoader.java`
- `src/main/java/main/model/levels/LevelManager.java`
- `src/main/java/main/model/entities/states/TriggerPlatformModel.java`
- `src/main/java/main/model/entities/states/TriggerSpikeModel.java`
- Deleted `src/main/java/main/model/entities/states/SpikeModel.java`

View rendering additions/updates:

- Added `src/main/java/main/view/render/PlayerRenderer.java`
- Reworked `src/main/java/main/view/render/LevelRenderer.java`
- Updated `src/main/java/main/view/GameView.java`

Result:

- No `Graphics/BufferedImage` imports remain in `main.model`.
- Model entities now hold state/data and behavior only.
- Rendering is performed by view renderers.

Step 7: Move State Classes to Controller Package

State-machine wrapper classes were moved from view package to controller package.

Added:

- `src/main/java/main/controller/state/GameBaseState.java`
- `src/main/java/main/controller/state/GamingState.java`
- `src/main/java/main/controller/state/MenuState.java`
- `src/main/java/main/controller/state/LeaderboardState.java`
- `src/main/java/main/controller/state/LevelSelectState.java`

Deleted:

- `src/main/java/main/view/interfaces/GameBaseState.java`
- `src/main/java/main/view/interfaces/GamingState.java`
- `src/main/java/main/view/interfaces/MenuState.java`
- `src/main/java/main/view/interfaces/LeaderboardState.java`
- `src/main/java/main/view/interfaces/LevelSelectState.java`

Controller import wiring updated in:

- `src/main/java/main/controller/Game.java`

Result:

- Screen state machine responsibility is now explicitly in controller layer.

Step 8: Remove Observer Pattern (Direct Call Flow)

Observer/event indirection was removed because it did not provide practical value in this codebase.

Changes:

- Deleted `src/main/java/main/model/observerEvents/GameObserver.java`
- Removed observer list + notify methods from `src/main/java/main/model/GameModel.java`
- Removed `Game implements GameObserver` and callbacks from `src/main/java/main/controller/Game.java`
- Added explicit side-effect handling in `Game.update()` via `handleModelSideEffects()`.
- Moved score persistence call to controller (`recordLevelCompletion()` in `Game`).

Result:

- Flow is now direct and simple:
 - `GameModel` updates state.
 - `Game` detects transitions and executes audio/persistence/transition side effects.
- This matches the requested "controller calls audio/persistence directly" model.

Current Architecture State

- Model:
 - Holds gameplay state/rules/collision progression.
 - No rendering methods.
 - No `java.awt.*/BufferedImage` usage in model package.
- View:
 - Renders via renderer classes.
 - Emits actions through callback interfaces.
- Controller:
 - Owns state machine and transitions.
 - Owns side effects (audio, score writes).
 - Handles input routing.

Concrete Examples (What Changed / What Moved)

1. Input no longer reaches into view internals.

- Before: key handlers directly called menu/leaderboard objects.
- Now: input uses action ports and controller routes it.
- Example:
 - `src/main/java/main/controller/inputs/KeyboardInputs.java:105`
 - `src/main/java/main/controller/inputs/KeyboardInputs.java:118`
 - `src/main/java/main/controller/Game.java:148`

2. Main menu no longer drives game flow directly.

- Before: menu called state transitions and quit itself.
- Now: menu emits callbacks and controller decides.
- Example:
 - `src/main/java/main/view/states/MainMenu.java:228`
 - `src/main/java/main/view/states/MainMenu.java:243`
 - `src/main/java/main/controller/Game.java:142`

3. Leaderboard file parsing moved out of view.

- Before: leaderboard view read and parsed score file.
- Now: view reads from a data source backed by service.
- Example:
 - `src/main/java/main/view/states/Leaderboard.java:17`
 - `src/main/java/main/model/leaderboard/LeaderboardService.java:15`
 - `src/main/java/main/controller/Game.java:110`

4. Player rendering moved from model to view renderer.

- Before: `Player` loaded sprite atlas and rendered itself.
- Now: `Player` is simulation-only; `PlayerRenderer` draws.
- Example:
 - `src/main/java/main/model/entities/entity/Player.java:33`
 - `src/main/java/main/view/render/PlayerRenderer.java:29`
 - `src/main/java/main/view/GameView.java:31`

5. Level entity rendering moved from model to `LevelRenderer`.

- Before: `Level` and entities had `draw*/render` methods.
- Now: model exposes data lists/getters; renderer draws all.
- Example:
 - `src/main/java/main/model/levels/Level.java:263`
 - `src/main/java/main/model/levels/Level.java:275`
 - `src/main/java/main/view/render/LevelRenderer.java:38`
 - `src/main/java/main/view/render/LevelRenderer.java:157`

6. State classes moved from view package to controller package.

- Before: `main.view.interfaces.*State`.
- Now: `main.controller.state.*`.
- Example:
 - `src/main/java/main/controller/state/MenuState.java:1`
 - `src/main/java/main/controller/state/GamingState.java:1`
 - `src/main/java/main/controller/Game.java:20`

7. Observer flow removed; controller now does direct side effects.

- Before: `GameModel` notified `GameObserver` callbacks.
- Now: `GameModel` updates flags/state; `Game` performs audio/persistence directly.

- Example:
 - `src/main/java/main/model/GameModel.java:37`
 - `src/main/java/main/model/GameModel.java:205`
 - `src/main/java/main/controller/Game.java:148`
 - `src/main/java/main/controller/Game.java:173`

Full Concrete Change Log (All Files Touched)

Added files

- `src/main/java/utilities/GameConfig.java`
 - Added centralized game/tile constants so model and utilities no longer depend on `main.controller.Game`.
- `src/main/java/main/model/leaderboard/ScoreEntry.java`
 - Added leaderboard DTO (`name`, `level`, `deaths`, `time`).
- `src/main/java/main/model/leaderboard/LeaderboardService.java`
 - Added score file parse/sort/filter service (moved out of view).
- `src/main/java/main/view/render/PlayerRenderer.java`
 - Added player sprite rendering in view layer.
- `src/main/java/main/controller/state/GameBaseState.java`
 - Added controller-side base class for screen states.
- `src/main/java/main/controller/state/GamingState.java`
 - Added controller-side gameplay state wrapper.
- `src/main/java/main/controller/state/MenuState.java`
 - Added controller-side menu state wrapper.
- `src/main/java/main/controller/state/LeaderboardState.java`
 - Added controller-side leaderboard state wrapper.
- `src/main/java/main/controller/state/LevelSelectState.java`
 - Added controller-side level-select state wrapper.

Deleted files

- `src/main/java/main/model/levels/LevelManagerHost.java`
 - Removed model-to-controller callback contract.
- `src/main/java/main/model/entities/states/SpikeModel.java`
 - Removed sprite-holding model state class.
- `src/main/java/main/model/observerEvents/GameObserver.java`
 - Removed observer interface after direct-call simplification.
- `src/main/java/main/view/interfaces/GameBaseState.java`
 - Deleted old view-package state class after move to controller package.
- `src/main/java/main/view/interfaces/GamingState.java`
 - Deleted old view-package state class after move to controller package.
- `src/main/java/main/view/interfaces/MenuState.java`
 - Deleted old view-package state class after move to controller package.
- `src/main/java/main/view/interfaces/LeaderboardState.java`
 - Deleted old view-package state class after move to controller package.
- `src/main/java/main/view/interfaces/LevelSelectState.java`

- Deleted old view-package state class after move to controller package.

Modified files

- `src/main/java/main/controller/Game.java`
 - Rewired dependencies and input attachment.
 - Implemented callback actions for passive views.
 - Switched to controller-side state classes.
 - Added direct side-effect handling (`handleModelSideEffects`) for death/respawn/transition.
 - Moved score persistence call to controller (`recordLevelCompletion`).
 - Removed observer implementation usage.
- `src/main/java/main/model/GameModel.java`
 - Removed observer list and notification methods.
 - Kept state update logic only.
 - Added simple `runCompleted` flag with `consumeRunCompleted`.
 - Exposed simple getters needed by controller (`getStartTime`, `getTotalDeaths`).
- `src/main/java/main/controller/facades/IGameActions.java`
 - Expanded controller input port with menu/leaderboard/mouse methods.
- `src/main/java/main/controller/facades/IGameRead.java`
 - Simplified read port to only what input needs (`getGameState`, `isEditingPlayerName`).
- `src/main/java/main/controller/inputs/KeyboardInputs.java`
 - Removed direct view access.
 - Mapped key events to action port methods only.
- `src/main/java/main/controller/inputs/MouseInputs.java`
 - Removed `GamePanel/Game` reach-through.
 - Forwarded raw mouse events to controller action port only.
- `src/main/java/main/view/GamePanel.java`
 - Became passive view surface.
 - Added `attachInputListeners(...)`.
 - Removed internal creation of input controllers.
- `src/main/java/main/view/GameWindow.java`
 - Fixed focus handling to notify controller via panel callback on focus lost.
- `src/main/java/main/view/GameView.java`
 - Switched rendering to dedicated view renderers (`LevelRenderer`, `PlayerRenderer`).
 - Removed rendering through model methods.
- `src/main/java/main/view/render/LevelRenderer.java`
 - Took over drawing of spikes, trigger spikes, trigger platforms, death sprites, spawn platform.
 - Uses model getters/data instead of model draw methods.
- `src/main/java/main/view/states/MainMenu.java`
 - Replaced direct game-state manipulation with `MainMenuActions`.
 - Added `Supplier<String>` for player name read.
- `src/main/java/main/view/states/LevelSelect.java`
 - Replaced direct game-state/level changes with `LevelSelectActions`.
- `src/main/java/main/view/states/Leaderboard.java`
 - Replaced direct file access with `LeaderboardDataSource`.
 - Uses `LeaderboardActions` for back/navigation intents.
- `src/main/java/main/model/levels/LevelManager.java`

- Removed host/controller callback dependency.
 - Simplified loading path and config apply calls.
 - `loadNextLevel()` returns boolean result instead of triggering controller.
- `src/main/java/main/model/levels/LevelConfigLoader.java`
 - Removed sprite/image coupling in config application.
 - Applies pure data config into model objects (IDs/numbers).
- `src/main/java/main/model/levels/Level.java`
 - Removed all `draw*` rendering methods.
 - Switched entity creation from image/sprite refs to sprite IDs.
 - Added model-side getters used by renderers (`getSpikes`, `getTriggerSpikes`, `getDeathSprites`, `getSpawnPlatform`).
 - Kept gameplay/collision/physics interactions only.
- `src/main/java/main/model/entities/entity/Entity.java`
 - Removed hitbox debug drawing.
 - Added simple geometry getters (`getX/getY/getWidth/getHeight`).
- `src/main/java/main/model/entities/entity/Player.java`
 - Removed sprite atlas loading/render method.
 - Kept simulation/animation-state progression only.
 - Updated death position recording to data-only call.
- `src/main/java/main/model/entities/entity/Spike.java`
 - Removed rendering and sprite image storage.
 - Stores sprite ID for view renderer.
- `src/main/java/main/model/entities/entity/TriggerSpike.java`
 - Removed rendering and image dependency.
 - Stores sprite ID and movement/collision logic only.
- `src/main/java/main/model/entities/entity/TriggerPlatform.java`
 - Removed rendering and image dependency.
 - Stores tile sprite IDs and movement/collision/audio trigger logic only.
- `src/main/java/main/model/entities/entity/SpawnPlatform.java`
 - Removed rendering/image field.
 - Kept animation state and movement behavior only.
- `src/main/java/main/model/entities/entity/MovingPlatform.java`
 - Removed rendering/image field.
 - Kept movement logic only.
- `src/main/java/main/model/entities/entity/DeathSprite.java`
 - Removed rendering/image field.
 - Kept position/size data only.
- `src/main/java/main/model/entities/states/TriggerPlatformModel.java`
 - Converted from `BufferedImage` sprite storage to integer sprite IDs + tile ID list.
- `src/main/java/main/model/entities/states/TriggerSpikeModel.java`
 - Converted from `BufferedImage` sprite storage to integer sprite ID.
- `src/main/java/main/model/entities/states/PlayerModel.java`
 - Updated config constant references to `GameConfig`.
- `src/main/java/utilities/HelpMethods.java`
 - Removed controller constant dependency; now uses `GameConfig`.
- `src/main/java/utilities/LoadSave.java`

- Removed controller constant dependency; now uses `GameConfig`.

Deep Concrete MVC Delta (Method-Level)

This section lists concrete API/method changes that were made to enforce MVC boundaries in code, not just by intent.

Controller-mediated flow (instead of view/model reach-through)

- `src/main/java/main/controller/Game.java:148`
 - Added `handleModelSideEffects()` to perform side effects (audio + transitions + score write trigger) in controller.
- `src/main/java/main/controller/Game.java:173`
 - Added `recordLevelCompletion()` in controller. Persistence now happens here via `LoadSave.appendToScoreFile(...)`, not in view/model.
- `src/main/java/main/controller/Game.java:271`
 - `setGameState(GameState newState)` now centralizes screen transitions and invokes model transition-entry hooks.
- `src/main/java/main/controller/Game.java:321`
 - `moveLeftPressed()` and other movement methods implement `IGameActions`, so input handlers call controller ports, not internals.
- `src/main/java/main/controller/Game.java:389`
 - `mouseMoved(int x, int y)` routes by active state and forwards to active view only.
- `src/main/java/main/controller/Game.java:416`
 - `onPlay()`, `onOpenLevelSelect()`, `onOpenLeaderboard()`, `onBackToMenu()`, `onSelectLevel(int)` implement passive-view callbacks.

Passive input ports (input -> interface -> controller)

- `src/main/java/main/controller/facades/IGameActions.java:3`
 - Expanded to contain controller action surface (`move*`, `jump*`, pause/menu, leaderboard nav, menu text input, mouse events).
- `src/main/java/main/controller/facades/IGameRead.java:5`
 - Reduced to read-only query surface needed by inputs (`getGameState()`, `isEditingPlayerName()`).
- `src/main/java/main/controller/inputs/KeyboardInputs.java:85`
 - `keyTyped(...)` sends text input through `actions.menuNameTyped(...)`.
- `src/main/java/main/controller/inputs/KeyboardInputs.java:93`
 - `keyPressed(...)` routes behavior via `actions`, including leaderboard navigation and menu editing control keys.
- `src/main/java/main/controller/inputs/KeyboardInputs.java:129`
 - `keyReleased(...)` only executes command bindings and action-port calls.
- `src/main/java/main/controller/inputs/MouseInputs.java:22`
 - `mouseMoved(...)` now only forwards coordinates to `actions.mouseMoved(...)`.
- `src/main/java/main/controller/inputs/MouseInputs.java:39`
 - `mousePressed(...)` and `mouseReleased(...)` only emit controller actions.

Passive view surface (panel/window)

- `src/main/java/main/view/GamePanel.java:24`
 - Added `attachInputListeners(...)` so listeners are composed externally by controller.
- `src/main/java/main/view/GamePanel.java:46`
 - `paintComponent(...)` delegates render to controller (`game.render(g)`), panel does not run game logic.
- `src/main/java/main/view/GameWindow.java:27`
 - Focus-loss callback wired to `gamePanel.onWindowFocusLost()` (controller path), removing incorrect focus-gained behavior.

View callbacks instead of game-state writes in UI classes

- `src/main/java/main/view/states/MainMenu.java:228`
 - `handleSelection(int)` now calls callbacks:
 - `actions.onPlay()`, `actions.onOpenLevelSelect()`, `actions.onOpenLeaderboard()`, `actions.onQuit()`.
- `src/main/java/main/view/states/MainMenu.java:260`
 - `handleNameKeyPressed(...)` sends confirm via `actions.onSetPlayerName(...)`.
- `src/main/java/main/view/states/LevelSelect.java:239`
 - `handleLevelSelection(int)` no longer mutates state directly; uses `actions.onSelectLevel(levelIndex)`.
- `src/main/java/main/view/states/LevelSelect.java:230`
 - Back button uses `actions.onBackToMenu()`.
- `src/main/java/main/view/states/Leaderboard.java:17`
 - Introduced `LeaderboardDataSource` contract so view does not read files.
- `src/main/java/main/view/states/Leaderboard.java:94`
 - Rendering reads `dataSource.loadEntriesForLevel(...)` supplied by controller/model service.

Rendering extracted from model into view renderers

- `src/main/java/main/view/GameView.java:25`
 - `GameView` now composes `LevelRenderer` + `PlayerRenderer`.
- `src/main/java/main/view/GameView.java:29`
 - `renderGame(...)` calls renderer classes; no model draw calls.
- `src/main/java/main/view/render/PlayerRenderer.java:29`
 - Added `render(Player player, Graphics g)` with atlas ownership in view.
- `src/main/java/main/view/render/LevelRenderer.java:38`
 - Added `renderBackgroundAndTerrain(...)` for terrain + dynamic entities.
- `src/main/java/main/view/render/LevelRenderer.java:81`
 - Added `renderObjectLayer(...)`.
- `src/main/java/main/view/render/LevelRenderer.java:101`
 - Added `renderSpawnPlatform(...)`.
- `src/main/java/main/view/render/LevelRenderer.java:125`
 - Added private render methods for each entity type (`renderSpike`, `renderTriggerSpike`, `renderTriggerPlatform`, `renderDeathSprite`).

Model entities/levels became simulation-only (no render methods)

- `src/main/java/main/model/entities/entity/Player.java:20`
 - `Player` now updates simulation and animation state only.
- `src/main/java/main/model/entities/entity/Player.java:59`
 - `die()` records death position via data call `currentLevel.recordDeathPosition(...)` and no sprite/render work.
- `src/main/java/main/model/entities/entity/Player.java:223`
 - View-facing animation state exposed via getters (`getPlayerAction()`, `getAniIndex()`), renderer consumes these.
- `src/main/java/main/model/entities/entity/Spike.java:5`
 - Stores `int spriteId`; no `BufferedImage`, no `render(...)`.
- `src/main/java/main/model/entities/entity/TriggerSpike.java:26`
 - Exposes `getSpriteId()` for renderer; movement/collision remain in model.
- `src/main/java/main/model/entities/entity/TriggerPlatform.java:52`
 - Exposes sprite IDs/tile positions (`getFirstTileSpriteId()`, `getTileSpriteIds()`), no draw methods.
- `src/main/java/main/model/entities/entity/SpawnPlatform.java`
 - Pure animation/collision state; no rendering logic.
- `src/main/java/main/model/entities/entity/MovingPlatform.java`
 - Pure movement update logic; no rendering logic.
- `src/main/java/main/model/entities/entity/DeathSprite.java`
 - Pure positional entity (size/position only).
- `src/main/java/main/model/entities/entity/Entity.java`
 - Hitbox debug draw removed; now geometry only.
- `src/main/java/main/model/levels/Level.java:297`
 - `createSpikesFromTile(...)` now builds spikes with sprite IDs, not sprite images.
- `src/main/java/main/model/levels/Level.java:321`
 - `createTriggerSpikesFromTile(...)` now builds trigger spikes with sprite IDs/config data.
- `src/main/java/main/model/levels/Level.java:263`
 - Renderer-facing getters added/kept: `getSpikes()`, `getTriggerSpikes()`, `getDeathSprites()`, `getSpawnPlatform()`.
- `src/main/java/main/model/levels/LevelManager.java:85`
 - `loadNextLevel()` returns `boolean` (advanced or final level) instead of calling controller callbacks.

Model state classes switched from images to IDs

- `src/main/java/main/model/entities/states/TriggerPlatformModel.java:29`
 - Replaced image storage with `firstTileSpriteId` and `List<Integer> tileSpriteIds`.
- `src/main/java/main/model/entities/states/TriggerSpikeModel.java:15`
 - Replaced image storage with `int spriteId`.
- `src/main/java/main/model/levels/LevelConfigLoader.java:141`
 - `applyConfig(...)` now applies numeric IDs/offsets/speeds; no image objects pass into model.

Screen-state classes moved to controller package

- Added controller-owned state wrappers:
 - `src/main/java/main/controller/state/GameBaseState.java`
 - `src/main/java/main/controller/state/GamingState.java`
 - `src/main/java/main/controller/state/MenuState.java`
 - `src/main/java/main/controller/state/LeaderboardState.java`
 - `src/main/java/main/controller/state/LevelSelectState.java`
- Deleted view-package state wrappers:
 - `src/main/java/main/view/interfaces/GameBaseState.java`
 - `src/main/java/main/view/interfaces/GamingState.java`
 - `src/main/java/main/view/interfaces/MenuState.java`
 - `src/main/java/main/view/interfaces/LeaderboardState.java`
 - `src/main/java/main/view/interfaces/LevelSelectState.java`

Observer removal and direct call flow

- Deleted:
 - `src/main/java/main/model/observerEvents/GameObserver.java`
- Removed from model:
 - Observer list + notify methods in `src/main/java/main/model/GameModel.java`.
- Removed from controller:
 - `Game` implements `GameObserver` and callback indirection.
- New direct flow now:
 1. `Game.update()` -> `model.update()`
(`src/main/java/main/controller/Game.java:143`).
 2. `Game.handleModelSideEffects()` reads model/player flags
(`src/main/java/main/controller/Game.java:148`).
 3. Controller directly calls audio (`playDead`, `playRespawn`, `playNextLevel`) and persistence (`recordLevelCompletion`).
 4. Transition completion handled via `model.consumeRunCompleted()` -> `setGameState(MENU)` (`src/main/java/main/controller/Game.java:167`).

Concrete examples of "moved responsibility"

1. Player drawing responsibility moved:
 - From: `main.model.entities.entity.Player` (render/sprite concerns in model)
 - To: `main.view.render.PlayerRenderer.render(...)`
(`src/main/java/main/view/render/PlayerRenderer.java:29`)
2. Spike/trigger/platform drawing moved:
 - From: model entities/level draw methods
 - To: `LevelRenderer.renderSpike/renderTriggerSpike/renderTriggerPlatform`
(`src/main/java/main/view/render/LevelRenderer.java:125`)
3. Menu actions moved:
 - From: view directly transitioning game state

- To: callbacks `MainMenuActions` implemented in controller
(`src/main/java/main/controller/Game.java:416`)

4. Leaderboard file read moved:

- From: `Leaderboard` view doing file parsing
- To: `LeaderboardService.loadEntriesForLevel(...)`
(`src/main/java/main/model/leaderboard/LeaderboardService.java:11`)
- View now receives data via `LeaderboardDataSource`
(`src/main/java/main/view/states/Leaderboard.java:17`)

5. Transition side effects moved to direct controller calls:

- From: observer callback indirection
- To: `Game.handleModelSideEffects() + recordLevelCompletion()`
(`src/main/java/main/controller/Game.java:148`)